

PROJECT REPORT

Autonomous Robot Navigation in Webots



Ostbayerische Technische Hochschule
Amberg-Weiden

MAPPING, PERCEPTION, PATH PLANNING AND NAVIGATION

Submitted by

Team

Course

Autonomous Robots

Ostbayerische Technische Hochschule Amberg-Weiden

Department of Electrical Engineering, Media and Computer Science

July 9, 2026

Abstract

This report documents the design and implementation of an autonomous navigation controller for a RosBot mobile robot in the Webots simulator. The robot must first reach the blue pillar and then the yellow pillar while avoiding green hazardous ground, normal obstacles, narrow passages, too-narrow passages and floating wall structures. The solution uses only onboard sensors and avoids Supervisor ground truth. The controller is organised into hardware and odometry, perception, occupancy-grid mapping, global A* planning, local Dynamic Window Approach motion control, reactive safety, replanning and mission-level state control.

1 Project Objective

The goal of the project is to implement autonomous navigation for a RosBot in Webots. The robot starts in an unknown maze and must complete the mission in the following order:

1. build a useful map using onboard sensors,
2. detect and localise the blue and yellow pillars,
3. navigate from the start area to the blue pillar,
4. after reaching blue, navigate to the yellow pillar,
5. avoid the green ground region and all physical obstacles.

The implementation is designed for multiple maze files, so the robot should work even when it is placed at a different initial pose. The robot does not assume a predefined map and

does not use Webots Supervisor position data.

2 Project Structure

The codebase is divided into clear layers. Each file has one main responsibility, which makes the controller easier to debug and extend.

File	Main role
main_maze3.py	Creates robot, perception, navigator and mission objects.
my_robot.py	Device setup wrapper, odometry, coordinate conversion and low-level motion.
setup.py	Enables motors, sensors, cameras, lidar and IMU devices.
perception.py	Interprets RGB, depth and lidar data.
map.py	Maintains the occupancy grid, frontiers and path-planning map.
astar_2_spline.py	Runs A* search and smooths the final path.
navigation.py	Executes plan-follow-replan navigation with safety checks.
mission.py	Controls initial scan, exploration and final blue-to-yellow traversal.
CONSTANTS.py	Stores tunable parameters and map marker values.
utils.py	Provides helper functions such as colour segmentation and Bresenham lines.

Table 1: Main files used in the autonomous robot controller.

The entry point follows a clean construction sequence:

```
def main():
    robot = MyRobot()
    perception = Perception(robot)
    navigator = Navigator(robot,
        perception)
    mission = Mission(robot,
        perception, navigator)
    success = mission.run()
```

3 System Architecture

The controller follows a layered architecture. The hardware layer reads sensors and controls the motors. The perception layer converts raw camera and lidar data into useful information such as coloured pillars, green hazards and depth obstacles. The map layer stores this information in an occupancy grid. The navigation layer plans and follows paths, while the mission layer decides which phase the robot is currently executing.

Layer	Responsibility
Hardware / pose	Sensor access, wheel motors, encoder odometry and compass heading.
Perception	HSV colour detection, pillar localisation, green-floor detection and depth-obstacle projection.
Mapping	Log-odds occupancy grid, special cell marking, frontier detection and obstacle inflation.
Planning	Global A* path search with wall-clearance penalty and B-spline smoothing.
Navigation	Waypoint following, Dynamic Window Approach, emergency checks and replanning.
Mission	Initial scan, exploration, pillar discovery and final traversal.

Table 2: Layered system architecture.

A key design decision is that all Webots API calls remain on the main control thread. Sensor reading, motor commands and `robot.step()` are not executed from background threads, which avoids unstable behaviour in the Webots controller.

4 Robot Hardware and Sensors

The robot uses the RosBot platform in Webots. Four wheel motors are controlled in velocity

mode. Four wheel position sensors provide encoder values for odometry. An RGB camera is used for colour segmentation, while a depth camera is used to estimate object distance and project hazards into the map. The 2D lidar is used for obstacle mapping, and short-range sensors are used for emergency collision checks.

Device	Use
Wheel encoders	Estimate travelled distance.
Compass / IMU	Provide heading for odometry and turning.
Lidar	Detect walls for occupancy-grid mapping.
RGB camera	Detect blue, yellow, green and red regions.
Depth camera	Estimate range to pillars and detect obstacles missed by lidar.
Range sensors	Stop the robot when an obstacle is very close.

Table 3: Sensors used by the controller.

5 Odometry and Coordinates

The robot does not use ground-truth coordinates. Its pose is estimated from wheel encoders and compass heading. The encoder values are averaged on the left and right side of the robot, converted to wheel displacement, and then integrated using the current heading.

$$d = \frac{d_l + d_r}{2} \quad (1)$$

$$x_{t+1} = x_t + d \cos(\theta), \quad (2)$$

$$y_{t+1} = y_t + d \sin(\theta). \quad (3)$$

The map is a **300×300** grid representing a **10**

m×10 m area. Therefore each cell represents approximately:

$$\text{resolution} = \frac{10.0}{300} = 0.0333 \text{ m/cell} \quad (4)$$

The robot starts near the centre of the grid. World coordinates are converted to map coordinates before planning, and map coordinates are converted back into world coordinates when required for movement.

6 Occupancy Grid Mapping

The map is stored as a grid of cell values. Normal obstacles, unknown cells, free space, pillars, green hazards, temporary closures and floating walls have different values so that they can be visualised and handled differently during planning.

Cell type	Value	Meaning
Free space	0	Traversable area
Obstacle	1	Wall or obstacle
Unknown	255	Not observed yet
Blue pillar	100	Blue target marker
Yellow pillar	150	Yellow target marker
Green carpet	190	Forbidden ground
Closed cell	200	Blocked recovery area
Floating wall	210	Depth-only obstacle

Table 4: Important occupancy-grid marker values.

For each lidar endpoint, a Bresenham line is drawn from the robot cell to the hit cell. Cells before the endpoint are updated as free space, while the final cell is updated as occupied. Log-odds values are used so repeated observations increase the confidence of the map.

```
for map_target in lidar_map_points
:
```

```
points = bresenham_line(
    robot_pos, map_target)
for x, y in points[:-1]:
    log_odds[y, x] -= 0.36
    # free cells
x, y = points[-1]
log_odds[y, x] += 0.85
    # obstacle cell
```

Special cells such as green carpet and floating walls are protected. This means that later lidar readings cannot accidentally erase hazards or depth-detected obstacles.

7 Perception

The perception module converts sensor readings into semantic information. It uses HSV colour segmentation because colour thresholds are easier to tune in HSV than in RGB.

Pillar detection

Blue and yellow pillars are detected by thresholding their HSV colour masks. If the coloured blob has enough pixels, the object is accepted as a pillar. The horizontal centroid gives the bearing and the median depth over the blob gives the range.

$$x_r = r \cos(\beta), \quad (5)$$

$$y_r = r \sin(\beta) \quad (6)$$

Here r is the median depth and β is the bearing from the image centre. The local point is transformed into the world frame using the robot pose and then stored as a map coordinate.

Green hazard detection

The green area is a forbidden region. The robot checks the lower central part of the RGB image for green pixels and uses depth to verify that the green region is close. Detected green pixels are projected to the ground plane and stamped into the map as `GREEN_CARPET`. During navigation these cells are treated as hard obstacles.

Depth obstacle detection

A horizontal lidar can miss objects that are too low, flat on the floor or floating above the scan plane. The depth camera helps detect such obstacles by projecting depth points into the ground plane. Ground-level obstacles are marked as normal obstacles, while elevated obstacles are marked as floating walls.

8 Exploration Strategy

At the start, the robot has no complete map. The mission first performs a slow in-place scan. The 360-degree turn is divided into small slices, and the robot waits for the sensors to settle before adding lidar data to the grid. This produces a cleaner initial map.

After that, the robot uses frontier exploration. A frontier is a free cell next to an unknown cell. Frontier clusters represent useful places to explore because they lie at the boundary of known and unknown space.

```
regions = map.detect_frontiers()
for region in regions:
    centroid = mean(region)
    if planner(robot_position,
               centroid):
        navigate_to(centroid)
```

The robot continues selecting reachable frontier centroids until both pillars are localised or the exploration budget is exhausted.

9 A* Path Planning

Global path planning is performed using A* search on the occupancy grid. The planner uses eight-connected motion with two-pixel steps. Straight moves are cheaper than diagonal moves, and the heuristic is based on Euclidean distance.

$$f(n) = g(n) + h(n) \quad (7)$$

The planner also adds a clearance penalty near walls. This makes the robot prefer routes that are slightly longer but safer, instead of driving directly beside obstacles.

$$C_{penalty}(n) = \begin{cases} k \left(1 - \frac{d(n)}{d_{safe}}\right), & d(n) < d_{safe} \\ 0, & d(n) \geq d_{safe} \end{cases} \quad (8)$$

After the path is found, B-spline smoothing is applied. The smoothed path is easier for the local controller to follow because it reduces sharp grid-based turns.

10 Navigation and Replanning

The navigation module is the executor. It receives a goal cell from the mission, asks the planner for a path, converts the dense path into waypoints, faces the first waypoint, and then follows the path.

```

for attempt in range(
    NAV_MAX_REPLANS):
    start = robot.get_map_position
        ()
    path = planner(start, goal)
    if not path:
        nudge_rotate()
        continue
    waypoints = to_waypoints(path)
    face_path(waypoints)
    result = follow_waypoints(
        waypoints)
    if result == "reached":
        return "reached"
    # otherwise update map and
    replan

```

This plan–follow–replan strategy is important because the map changes while the robot moves. If a new obstacle, green patch or floating wall is detected, the current path becomes invalid and a new path must be planned.

11 Dynamic Window Approach

The local motion controller uses a Dynamic Window Approach style method. Several linear and angular velocity candidates are sampled. Each candidate predicts a short future motion and receives a score based on heading alignment, distance to the target and speed. The best candidate is converted into left and right wheel velocities.

$$v_l = \frac{v - \omega L/2}{R}, \quad (9)$$

$$v_r = \frac{v + \omega L/2}{R} \quad (10)$$

Here v is the linear velocity, ω is angular ve-

locity, L is axle length and R is wheel radius.

12 Reactive Safety

Reactive safety is checked during every navigation tick. The navigator stops and requests replanning when a dangerous condition is detected.

- green ground is close in front of the robot,
- lidar detects an obstacle inside the front cone,
- range sensors detect a near obstacle,
- depth camera detects an obstacle at collision height,
- the robot is stuck and no longer making progress.

When the robot is stuck, the map can stamp the blocked region in front of the robot. The next A* search then avoids that same location. Recovery mainly uses in-place rotation instead of reversing, so the robot changes viewpoint and tries a new path safely.

13 Mission Flow

The high-level mission follows a simple state machine:

1. **INIT_SCAN**: rotate in place, build the first map and search for pillars.
2. **EXPLORE**: move to reachable frontier regions until both pillars are known.
3. **TRAVERSE**: drive to the blue pillar first and then to the yellow pillar.
4. **DONE**: stop the motors and keep the simulation stable.

The pillars can be discovered in any order during exploration. However, the final traversal

always follows the required order: **start** → **blue** → **yellow**.

14 Important Parameters

Parameter	Value	Purpose
TIME_STEP	32 ms	Webots step
MAP_SIZE	300	Grid size
RESOLUTION	0.0333 m	Cell size
WHEEL_RADIUS	0.043 m	Wheel model
AXLE_LENGTH	0.18 m	Differential drive
NAV_MAX_REPLANS	20	Replanning limit
NAV_WAYPOINT_STRIDE	6	Path down-sampling
SAFE_STOP_DIST_M	0.13 m	Lidar stop
SAFE_RANGE_STOP_M	0.06 m	Range stop
COLUMN_MIN_PIXELS	120	Pillar blob gate
INITIAL_SCAN_SLICES	24	Initial scan parts

Table 5: Selected configuration values used by the controller.

15 Testing Plan

The controller should be tested on all five maze files. The main evaluation criteria are whether the robot reaches the blue pillar, reaches the yellow pillar after blue, avoids the green area, avoids floating walls and finishes in a reasonable simulation time.

Maze	Blue	Yellow	Green avoided
Maze 1	To test	To test	To test
Maze 2	To test	To test	To test
Maze 3	To test	To test	To test
Maze 4	To test	To test	To test
Maze 5	To test	To test	To test

Table 6: Suggested result table for final evaluation.

16 Conclusion

This project implements a complete autonomous navigation pipeline for a RosBot robot in

Webots. The system builds an occupancy grid, detects blue and yellow pillars, marks green hazardous ground, explores unknown space using frontiers, plans global paths using clearance-aware A*, follows paths using local motion control and replans whenever the environment becomes unsafe. The modular design keeps perception, mapping, planning, navigation and mission logic separate, which makes the solution easier to understand, test and improve.

AI Usage Declaration

Generative AI tools were used for report structuring, LaTeX formatting, grammar improvement and summarising the provided project source files. The technical content was adapted from the submitted autonomous robot implementation.

References

- [1] P. E. Hart, N. J. Nilsson, and B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, 1968.
- [2] D. Fox, W. Burgard, and S. Thrun, “The Dynamic Window Approach to Collision Avoidance,” *IEEE Robotics & Automation Magazine*, 1997.
- [3] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
- [4] Cyberbotics, *Webots User Guide and Reference Manual*.